defimoon.org
twitter.com/Defimoon_org
linkedin.com/company/defimoon

Rome, Italy, 00165

Smart contract's
Audit report

StorX

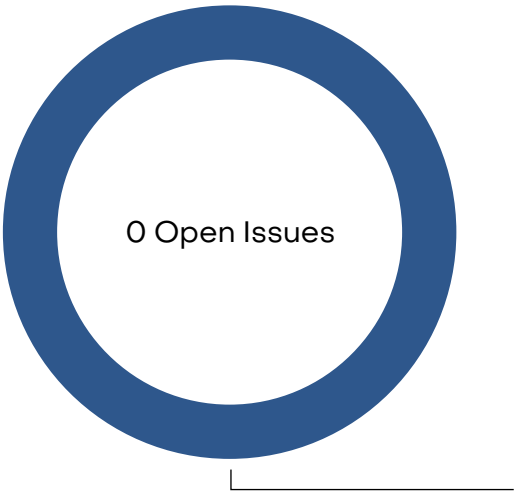
June, 2025

DEFIMOON
be secure

16 June 2025
This audit report was prepared by DefiMoon for StorX project.

Audit information

Description	StorX Token smart contract with Proxy upgradable pattern
Audited files	Proxy.flat.sol, StorxToken.flat.sol
Timeline	12 June 2025 – 16 June 2025
Approved by	Artur Makhnach, Kirill Minyaev
Languages	Solidity
Methods	Architecture Review, Unit Testing, Functional Testing, Manual Review
Source code	- https://github.com/StorXNetwork/storx-contracts/blob/f886b8c1c75228c11d024683f909cd06bc335683/flats/Proxy.flat.sol - https://github.com/StorXNetwork/storx-contracts/blob/f02aebb0f77898bb055ba450c91975e6c67c207c/flats/StorxToken.flat.sol
Status	Passed



Disclaimer

This audit is not financial, investment, or any other kind of advice and could be used for informational purposes only. This report is not a substitute for doing your own research and due diligence should always be paid in full to any project. Defimoon is not responsible or liable for any loss, damage, or otherwise caused by reliance on this report for any purpose. Defimoon has based this audit report solely on the information provided by the audited party and on facts that existed before or during the audit being conducted. Defimoon is not responsible for any outcome, including changes done to the contract/contracts after the audit was published. This audit is fully objective and only discerns what the contract is saying without adding any opinion to it. Defimoon has no connection to the project other than the conduction of this audit and has no obligations other than to publish an objective report. Defimoon will always publish its findings regardless of the outcome of the findings. The audit only covers the subject areas detailed in this report and unless specifically stated, nothing else has been audited. Defimoon assumes that the provided information and materials were not altered, suppressed, or misleading. This report is published by Defimoon, and Defimoon has sole ownership of this report. Use of this report for any reason other than for informational purposes on the subjects reviewed in this report including the use of any part of this report is prohibited without the express written consent of Defimoon. In instances where an auditor or team member has a personal connection with the audited project, that auditor or team member will be excluded from viewing or impacting any internal communication regarding the specific audit.

Audit Information

Defimoon utilizes both manual and automated auditing approach to cover the most ground possible. We begin with generic static analysis automated tools to quickly assess the overall state of the contract. We then move to a comprehensive manual code analysis, which enables us to find security flaws that automated tools would miss. Finally, we conduct an extensive unit testing to make sure contract behaves as expected under stress conditions.

In our decision making process we rely on finding located via the manual code inspection and testing. If an automated tool raises a possible vulnerability, we always investigate it further manually to make a final verdict. All our tests are run in a special test environment which matches the "real world" situations and we utilize exact copies of the published or provided contracts.

While conducting the audit, the Defimoon security team uses best practices to ensure that the reviewed contracts are thoroughly examined against all angles of attack. This is done by evaluating the codebase and whether it gives rise to significant risks. During the audit, Defimoon assesses the risks and assigns a risk level to each section together with an explanatory comment.

Audit overview

No critical issues have been identified

This audit covers the core smart contract architecture of the StorX token ecosystem, specifically focusing on the following two components:

- **StorxToken** – an upgradable ERC-20–like token contract featuring minting, burning, blacklisting, role-based admin control, and initialization logic.
- **Proxy** – a minimal upgradeability proxy based on the legacy ZOS pattern used to delegate execution to the logic contract.

The objective of this audit is to evaluate the correctness and security of the token's role-based access control, mint/burn/blacklist mechanisms, and upgradeability logic. We also assess the structural integrity and upgrade safety of the proxy + logic contract pair and ensure alignment with ERC-20 conventions and best practices.

Scope

- Initialization and upgrade safety of the proxy/logic system.
- Role assignment and privilege overlap risks.
- Blacklist enforcement logic and admin control paths.
- Token minting behavior, constraints, and event emission.
- Upgradeability pattern compliance and compatibility with modern tooling.
- Storage layout collisions and initialization bypass potential.
- Code quality, gas usage, and maintainability under current Solidity standards.

General Assessment at Reaudit

The StorX token contracts **have been battle-tested in production for over five years**, and their upgrade to the new implementation follows the same proven proxy pattern that has served the project reliably since launch. Although the codebase must remain on Solidity 0.4.24 to maintain full compatibility with the original proxy, our re-audit confirms that this constraint **does not introduce new security risks** given the project's well-defined operational procedures and restricted role management.

The **team has proactively addressed every practical issue that could be resolved within the legacy framework** - correcting event logs, improving admin rotation, and aligning ERC-20 mint events with ecosystem expectations. These refinements, undertaken voluntarily after the initial audit, **demonstrate a strong commitment to user safety and long-term maintainability**.

In short, the updated implementation is **fit for continued production use**, and the community can be confident that the contracts operate as intended while preserving the project's solid five-year track record.

Summary of findings

ID	Description	Severity	Status
DFM-1	Un-protected Proxy Init	Medium	Mitigated
DFM-2	Un-capped Minting	Low	Mitigated
DFM-3	Bad OperatorTransferred Event	Low	Resolved
DFM-4	Self-Only setAdmin	Info	Resolved
DFM-5	Missing Initial Transfer Event	Low	Resolved
DFM-6	Legacy 0.4.24 + ZOS Proxy	Low	Acknowledged
DFM-7	Role Overlap Freeze-Risk	Low	Mitigated
DFM-8	upgradeToAndCall Silent Fail	Low	Acknowledged
DFM-9	Re-initialisation On Upgrade	Medium	Acknowledged
DFM-10	Public initializeAdmin	Medium	Mitigated
DFM-11	Minor Gas Inefficiencies	Info	Acknowledged

Application security checklist

Compiler errors	Passed
Possible delays in data delivery	Passed
Timestamp dependence	Passed
Integer Overflow and Underflow	Passed
Race Conditions and Reentrancy	Passed
DoS with Revert	Passed
DoS with block gas limit	Passed
Methods execution permissions	Passed
Private user data leaks	Passed
Malicious Events Log	Passed
Scoping and Declarations	Passed
Uninitialized storage pointers	Passed
Arithmetic accuracy	Passed
Design Logic	Passed
Cross-function race conditions	Passed

Detailed Audit Information

Contract Programming

Program version not specified	Passed
Program version too old	Passed
Integer overflow/underflow	Passed
Function input parameters lack of check	Passed
Function input parameters check bypass	Passed
Function access control lacks management	Passed
Critical operation lacks event log	Passed
Human/contract checks bypass	Passed
Random number generation/use vulnerability	Passed
Fallback function misuse	Passed
Race condition	Passed
Logical vulnerability	Passed
Other programming issues	Passed

Code Specification

Visibility not explicitly declared	Passed
Variable storage location not explicitly declared	Passed
Use keywords/functions to be deprecated	Passed
Other code specification issues	Passed

Gas Optimization

Assert misuse	Passed
High consumption loop	Passed
High consumption storage	Passed
“Out of Gas” Attack	Passed

Findings

DFM-1 « Un-protected Proxy Init»

Severity: Medium

Status: Mitigated

Description: The initialize() function located in StorXToken.sol is designed to mint the full token supply, assign the owner, operator, and set metadata such as name and symbol. However, this function is marked public and only protected by an initialized boolean flag stored in the logic contract's storage layout. When deployed through a proxy (Proxy.sol), the storage context resides in the proxy, not the logic contract—meaning unless initialize() is called *immediately* after deployment via script, any external user can call it first and seize control.
This creates a race condition where a forgotten or front-run initialize() call could result in a complete hijack of ownership and token supply.

Recommendation: Make initialize() callable only once per proxy instance by integrating OpenZeppelin's initializer modifier with correct storage isolation. Ensure deployment scripts enforce atomic initialization (deploy + initialize in the same transaction via upgradeToAndCall() or a factory contract).

Team Comments: It is deployed using script where both the contracts are deployed together and the initialize function is called, our proxy contract is deployed and running for more than 5 years, need to update the implementation contract address after deploying it.

Defimoon's Comments: This usage is valid and sane, just be mindful of potential front-running attacks, use private RPC (avoids public mempool) at deployment if possible for your chain.

DFM-2 « Un-capped Minting»

Severity: Low

Status: Mitigated

Description: The mint(address _to, uint256 _amount) function located in StorXToken.sol is restricted to the operator role, but imposes no upper limit on the amount that can be minted. Once assigned, an operator may mint any quantity of tokens at any time, which could lead to unchecked inflation of the totalSupply state variable.
While this may be intentional for administrative flexibility, it creates potential monetary policy risk and trust issues if not explicitly addressed in documentation.

Recommendation: If unlimited minting is intended, explicitly document this behavior in project specifications and community-facing materials. Otherwise, consider implementing a maximum supply cap or governance-controlled minting scheme.

Team Comments: The staking contract has this right as operator, and the owner won't update that address and the key of the owner address is stored securely in an offline machine.

Defimoon's Comments: Team was made aware of the risk exposure and mitigated by using trusted owner and key storage device.

DFM-3 « Bad OperatorTransferred Event»

Severity: Low

Status: Resolved

Description: The transferOperator(address _newOperator) function in StorXToken.sol emits an OperatorTransferred event. However, the event logs the previousOperator field using address(0) instead of the actual previous operator address (_operator). This creates misleading logs and obstructs accurate off-chain indexing and historical tracking of operator transitions.

Recommendation: Update the event emission line inside transferOperator() to use the correct reference:
emit OperatorTransferred(_operator, _newOperator);

Team Comments: Fixed, updated the emit statement.

Defimoon's Comments: In _transferOperator() we now see emit OperatorTransferred(_operator, newOperator_); before _operator is updated – event now accurate, issue resolved.

DFM-4 « Self-Only setAdmin»

Severity: Info

Status: Resolved

Description: The setAdmin() function in Proxy.sol restricts admin assignment such that the new admin can only be the msg.sender. This prevents the current admin from assigning a third party or rotating admin privileges to a new operational address unless that address performs the transaction directly. While this may be a safety mechanism, it can hinder flexibility during admin transitions, especially in emergency or automated scenarios such as compromised/lost admin EOA wallet.

Recommendation: Consider updating setAdmin() to accept a newAdmin parameter, and restrict the call to the current admin only (e.g., require(msg.sender == _admin)), which enables admin rotation without requiring new admin to pre-sign or call the function.

Team Comments: Fixed, updated the setAdmin function, to accept a new address as admin.

Defimoon's Comments: New signature setAdmin(address newAdmin) with require(newAdmin != address(0)) and event AdminUpdated(prev,new). Works as intended, issue resolved.

DFM-5 « Missing Initial Transfer Event»

Severity: Low

Status: Resolved

Description: The initialize() function in StorXToken.sol function assigns the entire _totalSupply directly to the deployer's balance via balances[msg.sender] = _totalSupply; but fails to emit a Transfer(address(0), msg.sender, _totalSupply) event.
This omission breaks a key convention of ERC-20 token minting and may cause problems for token explorers, supply indexers, and third-party analytics tools that rely on the Transfer event to track supply creation and circulating balances.

Recommendation: Add an explicit Transfer(address(0), msg.sender, _totalSupply); event after setting the balance during initialization to maintain ERC-20 compliance and ensure compatibility with common indexers and token scanners.

Team Comments: Fixed, added the event emit.

Defimoon's Comments: Line emit Transfer(address(0), msg.sender, _totalSupply); added – ERC-20 mint event now emitted, issue resolved.

DFM-6 « Legacy 0.4.24 + ZOS Proxy»

Severity: Low

Status: Acknowledged

Description: Both StorxToken.sol and Proxy.sol are written in Solidity version *0.4.24* and use an older ZOS-based upgradable proxy pattern that predates the EIP-1967 standard. The legacy Proxy contract lacks standardized storage slots, such as implementation and admin being stored at deterministic, collision-safe positions.
Additionally, all arithmetic operations rely on the SafeMath library despite Solidity ≥0.8.0 providing built-in overflow checks, which adds unnecessary gas overhead.
While functional, this architecture is outdated and can present tooling compatibility issues with modern analysis, verification, and indexing tools.

Recommendation: Upgrade to Solidity ≥0.8.x and adopt the EIP-1967-compatible upgradeable proxy pattern provided by OpenZeppelin. This will improve maintainability, compatibility, and reduce bytecode complexity while allowing native overflow protection and better slot management.

Team Comments: The proxy and the old implementation contracts deployed 5 years back have been using version 0.4.24 so we have kept the new implementation contract with the same solidity version.

Defimoon's Comments: Team was made aware of the risk exposure, no direct security risk present.

DFM-7 « Role Overlap Freeze-Risk»

Severity: Low

Status: Mitigated

Description: In StorxToken.sol, the setAdmin() function is callable by any address with onlyAuthorized privileges, which includes the owner, operator, and the admin itself. Since the adminAddress controls blacklist enforcement via setBlacklistStatus(address,bool), this allows any of the three roles to re-assign admin powers and indirectly freeze or unblock any account.
This overlap in critical authority weakens the privilege separation model and increases the potential for abuse or accidental fund freezing.

Recommendation: Separate admin assignment powers from operator and owner roles, or introduce additional checks (e.g. multi-sig, cooldown period) for setAdmin() to reduce privilege escalation risk.

Team Comments: Both admin and owner addresses will be saved securely in an offline machine with single person access, so this won't be an issue.

Defimoon's Comments: Team was made aware of the risk exposure and mitigated by allowing privileged access to trusted person and using secure key storage device.

DFM-8 « upgradeToAndCall Silent Fail»

Severity: Low

Status: Acknowledged

Description: The upgradeToAndCall(address newImplementation, bytes data) function in Proxy.sol uses a low-level .call(data) to initialize the new logic contract. However, it only checks whether the call returned true or false and does not bubble up the actual revert reason from the target contract.
If the initialization logic fails (e.g., throws an error or reverts), the require(success) fails with no explanation, making debugging and error tracking difficult.

Recommendation: Use inline assembly to bubble up revert reasons from failed delegatecall invocations, or switch to OpenZeppelin's modern transparent proxy pattern which automatically propagates revert messages.

Team Comments: As proxy contract is already deployed 5 years back, can't make changes in proxy contract.

Defimoon's Comments: Team was made aware of the risk exposure, usage is sane and valid, no direct security threat.

DFM-9 « Re-initialisation On Upgrade»

Severity: Medium

Status: Acknowledged

Description: The contract uses an initializer modifier from Initializable.sol, which only guards against re-execution of initializers within a single contract's context. However, if a new implementation contract with a different initializer (e.g. initializeV2()) is deployed and upgraded to via upgradeToAndCall(), that new function is unguarded and callable because its own initialized flag starts as false.
This allows a malicious or misconfigured upgrade to re-mint tokens, reassign roles, or override prior state.

Recommendation: Use a centralized storage slot in the proxy to track initialization status across upgrades, or adopt OpenZeppelin's Initializable contract that implements cross-impl initialization locking.

Team Comments: Not applicable.

Defimoon's Comments: Team was made aware of the risk exposure, , as long as new implementation deployment is conducted from trusted source (the team), no security threat present.

DFM-10 « Public initializeAdmin»

Severity: Medium

Status: Mitigated

Description: The initializeAdmin(address _admin) function in StorxToken.sol is publicly accessible and only protected by a check that adminAddress == address(0). If deployment scripts forget to call this function post-deployment (or the transaction fails), any external user can call it and assign themselves as the contract's blacklist-controlling admin.
Although the function includes a check to prevent calls from the logic contract directly (i.e., require(address(this) != msg.sender)), it remains publicly callable through the proxy.

Recommendation: initializeAdmin() to be callable only by a known initialization actor (e.g., the owner or operator), or move its logic into the main initialize() function to ensure admin setup occurs atomically during contract initialization.

Team Comments: Called in the script used to deploy the contract so will be set after the deployment.

Defimoon's Comments: Team was made aware of the risk exposure, deployment script is used correctly and securely, no risk present.

DFM-11 « Minor Gas Inefficiencies»

Severity: Info

Status: Acknowledged

Description: Several components of the codebase incur unnecessary gas costs due to outdated patterns:

- SafeMath usage in a Solidity 0.8+ context (if upgraded)
- Constant string variables (name, symbol) are not immutable
- Redundant state read/write for operator transitions and balance updates

These patterns are functional but suboptimal for modern deployments.

Recommendation: If upgrading the compiler version, remove SafeMath and use Solidity-native overflow checks. Consider declaring name, symbol, and decimals as immutable (if constructor-style setup is adopted) and audit storage slot access to eliminate double writes.

Team Comments: Can't upgrade solidity version.

Defimoon's Comments: Team was made aware of the misoptimisations present, this informational finding cannot be resolved without full system update which doesn't make sense in current contract's usage, not an issue for production use.

Methodology

Manual Code Review

We prefer to work with a transparent process and make our reviews a collaborative effort. The goal of our security audits is to improve the quality of systems we review and aim for sufficient remediation to help protect users. The following is the methodology we use in our security audit process.

Vulnerability Analysis

Our audit techniques include manual code analysis, user interface interaction, and whitebox penetration testing. We look at the project's web site to get a high-level understanding of what functionality the software under review provides. We then meet with the developers to gain an appreciation of their vision of the software. We install and use the relevant software, exploring the user interactions and roles. While we do this, we brainstorm threat models and attack surfaces. We read design documentation, review other audit results, search for similar projects, examine source code dependencies, review open issue tickets, and investigate details other than the implementation.

Documenting Results

We follow a conservative, transparent process for analyzing potential security vulnerabilities and seeing them through successful remediation. Whenever a potential issue is discovered, we immediately create an Issue entry for it in this document, even though we have not yet verified the feasibility and impact of the issue. This process is conservative because we document our suspicions early even if they are later shown to not represent exploitable vulnerabilities. We follow a process of first documenting the suspicion with unresolved questions, then confirming the issue through code analysis, live experimentation, or automated tests. Code analysis is the most tentative, and we strive to provide test code, log captures, or screenshots demonstrating our confirmation. After this we analyze the feasibility of an attack in a live system to make a final decision.

Suggested Solutions

We search for immediate mitigations that live deployments can take, and finally we suggest the requirements for remediation engineering for future releases. The mitigation and remediation recommendations should be scrutinized by the developers and deployment engineers, and successful mitigation and remediation is an ongoing collaborative process after we deliver our report, and before the details are made public.

Appendix A — Finding Statuses

Resolved	Contracts were modified to permanently resolve the finding
Mitigated	The finding was resolved by other methods such as revoking contract ownership or updating the code to minimize the effect of the finding
Acknowledged	Project team is made aware of the finding
Open	The finding was not addressed